1. The following is the bubble sort implementation seen in class.

```
void bubbleSort(double[] list) {
    double tmp;
    for (int i = 0; i < list.length - 1; i++) {
        for (int j = 0; j < list.length - i - 1; j++) {
            if (list[j] > list[j + 1]) {
                tmp = list[j];
                list[j] = list[j + 1];
                list[j + 1] = tmp;
            }
        }
    }
}
```

It is called on the array `{5, 1, 4, 2, 8, 3}`. What does the array look like after the first two iterations of the outer loop (`i = 0` and `i = 1`) are complete?
Answer: _____

---

2. The following is the bubble sort implementation seen in class.

```
void bubbleSort(double[] list) {
    double tmp;
    for (int i = 0; i < list.length - 1; i++) {
        for (int j = 0; j < list.length - i - 1; j++) {
            if (list[j] > list[j + 1]) {
                tmp = list[j];
                list[j] = list[j + 1];
                list[j + 1] = tmp;
            }
        }
    }
}
```

It is called on the array `{7, 3, 9, 1, 6, 2}`. What does the array look like after the first two iterations of the outer loop (`i = 0` and `i = 1`) are complete?
Answer: _____

---

3. The following method merges two sorted arrays into one sorted array.

```
double[] merge(double[] x, double[] y) {
    double[] result = new double[x.length + y.length];
    int i = 0, j = 0, k = 0;
    while (i < x.length && j < y.length) {
        if (x[i] <= y[j]) result[k++] = x[i++];
        else result[k++] = y[j++];
    }
    while (i < x.length) result[k++] = x[i++];
    while (j < y.length) result[k++] = y[j++];
    return result;
}
```

How many times is the comparison `x[i] <= y[j]` evaluated when calling `merge(x, y)` with `x = {1, 4, 7, 9}` and `y = {2, 3, 10}`?
Answer: _____

---

4. The following method merges two sorted arrays into one sorted array.

```
double[] merge(double[] x, double[] y) {
    double[] result = new double[x.length + y.length];
    int i = 0, j = 0, k = 0;
    while (i < x.length && j < y.length) {
        if (x[i] <= y[j]) result[k++] = x[i++];
        else result[k++] = y[j++];
    }
    while (i < x.length) result[k++] = x[i++];
    while (j < y.length) result[k++] = y[j++];
    return result;
}
```

How many times is the comparison `x[i] <= y[j]` evaluated when calling `merge(x, y)` with `x = {3, 5, 8}` and `y = {1, 2, 6, 7, 11}`?
Answer: _____

---

5. Consider the following merge sort pseudocode, where `merge` merges the sorted range from `firstIndex` to `mid` and the sorted range from `mid+1` to `lastIndex` into one sorted range.

```
1  mergeSort( array )
2    mergeSortRec( array, 0, array.length-1 )
3  mergeSortRec( array, firstIndex, lastIndex )
4    if (firstIndex >= lastIndex)
5      return
6    mid = (firstIndex+lastIndex)/2
7    mergeSortRec(array, firstIndex, mid)
8    mergeSortRec(array, mid+1, lastIndex)
9    merge(array, firstIndex, mid, mid+1, lastIndex)
```

`mergeSort` is called on the array `{13, 6, 2, 7, 11, 4, 8, 15}`. What does the array look like once the call on line 7 of the *first* call to `mergeSortRec` completely finishes (every recursive call inside it included)?
Answer: _____

---

6. Consider the following merge sort pseudocode, where `merge` merges the sorted range from `firstIndex` to `mid` and the sorted range from `mid+1` to `lastIndex` into one sorted range.

```
1  mergeSort( array )
2    mergeSortRec( array, 0, array.length-1 )
3  mergeSortRec( array, firstIndex, lastIndex )
4    if (firstIndex >= lastIndex)
5      return
6    mid = (firstIndex+lastIndex)/2
7    mergeSortRec(array, firstIndex, mid)
8    mergeSortRec(array, mid+1, lastIndex)
9    merge(array, firstIndex, mid, mid+1, lastIndex)
```

`mergeSort` is called on the array `{9, 4, 12, 1, 10, 3, 5, 7}`. What does the array look like once the call on line 8 of the *first* call to `mergeSortRec` completely finishes (every recursive call inside it included)?
Answer: _____

---

7. The following is the partition method seen in class.

```
int partition(double[] array, int left, int right, int pivotIndex) {
    int pivotPos = right;
    swap(array, pivotIndex, pivotPos);
    double pivot = array[pivotPos];
    right--;
    while (right >= left) {
        while (array[left] < pivot) left++;
        while (right >= left && array[right] >= pivot) right--;
        if (right > left) swap(array, left, right);
    }
    swap(array, left, pivotPos);
    return left;
}
```

where `swap(array, i, j)` exchanges `array[i]` and `array[j]`. What does the array `a = {5, 9, 2, 8, 4, 7, 1}` look like after calling `partition(a, 0, 6, 0)`?
Answer: _____

---

8. The following is the partition method seen in class.

```
int partition(double[] array, int left, int right, int pivotIndex) {
    int pivotPos = right;
    swap(array, pivotIndex, pivotPos);
    double pivot = array[pivotPos];
    right--;
    while (right >= left) {
        while (array[left] < pivot) left++;
        while (right >= left && array[right] >= pivot) right--;
        if (right > left) swap(array, left, right);
    }
    swap(array, left, pivotPos);
    return left;
}
```

where `swap(array, i, j)` exchanges `array[i]` and `array[j]`. What does the array `a = {6, 11, 3, 9, 2, 10, 4}` look like after calling `partition(a, 0, 6, 0)`?
Answer: _____

---

9. All algorithms below are the implementations seen in class. They are used to sort, in ascending order, an array of N distinct elements that is **already sorted in ascending order**.
Which of the following are true (select one or more):

- ☐ the number of comparisons of insertion sort grows linearly with N
- ☐ the number of comparisons of selection sort grows linearly with N
- ☐ the number of comparisons of bubble sort grows linearly with N
- ☐ the number of comparisons of quicksort, with the first element as pivot, grows quadratically with N
- ☐ the number of comparisons of quicksort, with the middle element as pivot, grows quadratically with N

---

10. All algorithms below are the implementations seen in class. They are used to sort, in ascending order, an array of N distinct elements that is **sorted in descending order**.
Which of the following are true (select one or more):

- ☐ the number of comparisons of insertion sort grows linearly with N
- ☐ the number of comparisons of insertion sort grows quadratically with N
- ☐ the number of comparisons of merge sort grows linearithmically with N
- ☐ the number of comparisons of quicksort, with the first element as pivot, grows quadratically with N
- ☐ the number of comparisons of selection sort grows linearithmically with N

---

11. For each sorting algorithm below (implementations seen in class), mark whether it is stable and whether it is in-place.

Algorithm	Stable	In-place
Selection sort	☐	☐
Insertion sort	☐	☐
Merge sort	☐	☐
Quicksort	☐	☐

---

12. For each sorting algorithm below (implementations seen in class), mark whether it is adaptive and whether it is stable.

Algorithm	Adaptive	Stable
Bubble sort	☐	☐
Selection sort	☐	☐
Insertion sort	☐	☐
Quicksort	☐	☐

---

13. Consider the following classes.

```
class Student {
    String name;
    double gpa;
    Student(String name, double gpa) { this.name = name; this.gpa = gpa; }
}

class ByGpa implements Comparator<Student> {
    public int compare(Student a, Student b) {
        return Double.compare(b.gpa, a.gpa);
    }
}
```

The array below is sorted with insertion sort, using a `ByGpa` object to compare elements.

```
Student[] roster = { new Student("Ann", 3.2), new Student("Bo", 3.9),
                     new Student("Cy", 2.8), new Student("Dee", 3.2) };
```

In which order are the names after sorting?
Answer: _____

---

14. Consider the following class.

```
class Word implements Comparable<Word> {
    String text;
    Word(String text) { this.text = text; }

    public int compareTo(Word other) {
        return text.length() - other.text.length();
    }
}
```

The array below is sorted with merge sort, using `compareTo` to compare elements.

```
Word[] words = { new Word("pear"), new Word("fig"), new Word("banana"),
                 new Word("kiwi"), new Word("apple") };
```

In which order are the words after sorting?
Answer: _____

---

15. An array-based list starts empty, with an array of capacity 4. When an element is appended and the array is full, the list allocates a new array of **twice** the capacity and copies all elements into it.
After appending 20 elements, what is the capacity of the array, and how many element copies were made in total by the grow operations?
Capacity: _____ Copies: _____

---

16. An array-based list starts empty, with an array of capacity 3. When an element is appended and the array is full, the list allocates a new array whose capacity is **3 more** than the current one and copies all elements into it.
After appending 13 elements, what is the capacity of the array, and how many element copies were made in total by the grow operations?
Capacity: _____ Copies: _____

---

17. `list` is a `java.util.ArrayList` containing N elements. For each operation, mark its cost in the worst case (for `add(e)`, mark its amortized cost).

Operation	O(1)	O(N)
`list.get(i)`	☐	☐
`list.add(0, e)`	☐	☐
`list.remove(list.size() - 1)`	☐	☐
`list.indexOf(e)`	☐	☐
`list.add(e)`	☐	☐

---

18. `list` is a `java.util.ArrayList` containing N elements. For each operation, mark its cost in the worst case (for `add(e)`, mark its amortized cost).

Operation	O(1)	O(N)
`list.set(i, e)`	☐	☐
`list.remove(0)`	☐	☐
`list.contains(e)`	☐	☐
`list.size()`	☐	☐
`list.add(list.size() / 2, e)`	☐	☐

---

19. What does the following code fragment print?

```
List<String> list = new ArrayList<>();
list.add("a");
list.add("b");
list.add("c");
list.add("d");
list.add(1, "e");
list.remove(3);
list.add(list.size(), "f");
list.set(0, list.get(2));
System.out.println(list);
```

Answer: _____

---

20. What does the following code fragment print?

```
List<String> list = new ArrayList<>();
list.add("p");
list.add("q");
list.add("r");
list.add("s");
list.add("t");
list.remove(1);
list.add(0, "u");
list.add(2, list.remove(4));
System.out.println(list);
```

Answer: _____

---

21. What does the following code fragment print?

```
List<Integer> nums = new ArrayList<>(List.of(4, 8, 15, 16, 23));
Iterator<Integer> it1 = nums.iterator();
Iterator<Integer> it2 = nums.iterator();
it1.next();
it1.next();
System.out.print(it1.next() + " ");
System.out.print(it2.next() + " ");
System.out.print(it1.next());
```

Answer: _____

---

22. What does the following code fragment print?

```
List<String> words = new ArrayList<>(List.of("red", "green", "blue"));
Iterator<String> a = words.iterator();
a.next();
a.next();
Iterator<String> b = words.iterator();
System.out.print(b.next() + " " + a.next() + " ");
System.out.print(a.hasNext());
```

Answer: _____
